
Amrutam Senior React Assignment

Time Limit: 72 Hours (expected effort: 8–12 hours)

Problem Statement

Build a production-ready **Ayurvedic Super App** consisting of three independent modules:

1. Consultation
2. Shop
3. Health Records

You are free to use any public APIs or mock APIs.

Your focus should be on **architecture, scalability, performance, and developer experience**, not pixel-perfect UI.

Functional Requirements

Module 1 – Consultations

Implement a consultation booking system.

Features:

- Doctor Listing
- Search
- Filters
- Doctor Details
- Available Slots
- Booking Flow
- Upcoming Consultation

- Cancel Booking

Handle

- Slot conflicts
 - Expired slots
 - Double booking attempts
-

Module 2 – Shop

Implement a mini ecommerce module.

Features

- Product Listing
- Infinite Scroll
- Search
- Multi-filter
- Sorting
- Product Details
- Cart
- Quantity updates
- Wishlist
- Checkout Summary

Persist cart locally.

Module 3 – Health Records

Implement a patient timeline.

Each record should support

- Lab Report
- Prescription
- Consultation
- Vaccination
- Allergy

Features

- Timeline View
 - Filters
 - Search
 - Tags
 - Attachment Preview (images/PDF thumbnails)
 - Group by Month/Year
-

Technical Constraints

Use

- React Native
- TypeScript
- React Navigation

State management is your choice.

No boilerplate starters.

Performance Challenge

Your application should comfortably support

- 5,000 doctors
- 20,000 products
- 10,000 health records

without UI lag.

Use mocked/generated data.

Demonstrate

- Virtualized rendering
 - Memoization
 - Efficient state updates
 - Lazy loading
-

Offline First

Your application should continue functioning without internet.

Implement

- Cached API responses
 - Offline cart
 - Offline bookings (queued)
 - Automatic sync once internet returns
-

Reliability

Handle gracefully

- Slow network
 - API timeout
 - Random failures
 - Empty responses
 - Partial responses
 - Invalid JSON
 - Session expiration
-

Developer Experience

Build the project as if another developer will join tomorrow.

Expected

- Clean architecture
 - Modular code
 - Reusable components
 - Shared design system
 - Strong typing
 - Proper folder organization
 - Minimal duplication
-

Production Engineering

Implement

- Environment configuration
 - API abstraction layer
 - Logging utility
 - Error Boundary
 - Global Toast system
 - Theme support
 - Dark Mode
 - Accessibility support
-

Testing

Write meaningful tests covering:

- Business logic
 - Custom hooks
 - Utility functions
 - One end-to-end user flow
-

Bonus (Choose Any Three)

- Feature Flags
 - Remote Config
 - Biometric Authentication
 - Deep Linking
 - Push Notification handling
 - Localization (2 languages)
 - Performance monitoring
 - Crash reporting abstraction
 - Secure local storage
 - Background synchronization
-

Documentation

Provide a README explaining:

- Folder structure
 - Architectural decisions
 - State management choice
 - Performance optimizations
 - Offline strategy
 - Trade-offs made
 - Future improvements
-

Evaluation Criteria

Area	%
Application Architecture	20
Code Quality & Maintainability	20
Performance & Scalability	20
Offline & Error Handling	15
State Management & Data Flow	10
Testing	5
Documentation	5
UX, Accessibility & Polish	5

What We're Really Evaluating

This assignment is intentionally designed to surface senior engineering skills. We're looking for candidates who can:

- Design a scalable React Native architecture.
- Make thoughtful trade-offs and justify them.
- Build for reliability, not just happy paths.

- Optimize rendering and state management for large datasets.
- Write maintainable, production-quality code.
- Think beyond features to performance, offline support, testing, and developer experience.